
Mise en place d'une pipeline de données sur le cloud

Une approche serverless pour les données financières

Réalisé par :

Paul BALAFAI
Ahmadou NIASS
Jean Luc BATABATI
Samba SOW
Papa Amadou NIANG

Sous la supervision de :

Mme. Mously DIAW
Lead ML Engineer / Experte IA et MLOps

ISE2 2025/2026

Table des matières

Introduction	1
1 Cadre conceptuel et enjeux	2
1.1 Concepts fondamentaux	2
1.2 Enjeux économiques et technologiques	3
2 Description des données et méthodologie de conception	5
2.1 Description des données	5
2.1.1 Ticker	6
2.1.2 Datetime	6
2.1.3 Open	6
2.1.4 High	6
2.1.5 Low	6
2.1.6 Close	6
2.1.7 Volume	6
2.2 Méthodologie	7
2.2.1 Conceptualisation et Choix d'Architecture	7
2.2.2 Implémentation du Pipeline ETL	7
2.2.3 Gouvernance des Données et Sécurité (IAM)	8
2.2.4 Phase de Validation et de Visualisation	9
3 Architecture globale du pipeline	10
3.1 Présentation du schéma général	10
3.2 Description des flux	11
3.2.1 Composants principaux	11
3.3 Choix du fournisseur cloud	12
4 Limites, risques et évolutions potentielles	14
4.1 Limites actuelles de la solution	14
4.1.1 Limites liées à la granularité du pipeline de données	14
4.1.2 Risques liés à la dérive du schéma des données	14
4.1.3 Limites de performance et d'optimisation de l'ETL	14

4.1.4	Manque d'observabilité et de supervision opérationnelle	15
4.2	Perspectives d'évolution de la solution	15
4.2.1	Évolution vers une architecture quasi temps réel	15
4.2.2	Renforcement de la robustesse du schéma et de la qualité des données	15
4.2.3	Optimisation des performances et des coûts du pipeline ETL . . .	15
4.2.4	Mise en place d'une observabilité complète du pipeline	15
	Conclusion	17
	Bibliographie	19

Introduction

Avec la digitalisation des marchés financiers, les données boursières sont aujourd'hui produites en continu et en volumes massifs, sous des formes variées telles que les prix, les volumes, les indices ou encore les indicateurs techniques. À titre d'illustration, les marchés financiers mondiaux génèrent plusieurs téraoctets de données par jour, et le volume total de données numériques mondiales a atteint 64 zettaoctets en 2020, avec une projection à 181 zettaoctets d'ici 2025, selon IDC (International Data Corporation). Cette explosion des données rend leur exploitation indispensable pour l'analyse financière, la gestion des risques et la prise de décision stratégique.

Dans ce contexte, les acteurs financiers investissent massivement dans les technologies analytiques : plus de 95% des institutions financières utilisent aujourd'hui des solutions Big Data, et la capacité à traiter efficacement des données historiques profondes constitue un avantage concurrentiel majeur. Cependant, les architectures traditionnelles basées sur des serveurs locaux montrent rapidement leurs limites en termes de scalabilité, de coûts et d'automatisation.

La problématique de notre projet est donc la suivante : comment concevoir un pipeline de données fiable, scalable et automatisé permettant de traiter des données boursières en mode batch sur une infrastructure cloud ? L'objectif principal est de mettre en place un pipeline Big Data de type ETL capable de collecter, nettoyer, transformer et stocker ces données de manière efficace, tout en garantissant performance, robustesse et traçabilité.

Pour répondre à cette problématique, nous avons conçu une architecture ETL basée sur le cloud, intégrant l'ensemble des étapes du pipeline, depuis l'ingestion des données jusqu'à leur mise à disposition pour l'analyse. Cette architecture sera présentée de manière synthétique dans la suite de cet exposé.

1 | Cadre conceptuel et enjeux

1.1 Concepts fondamentaux

Big Data : Le Big Data désigne l'ensemble des technologies permettant de traiter des données caractérisées par leur volume, leur vitesse et leur variété. Dans le domaine boursier, il s'agit principalement de grandes quantités de données financières (prix, volumes, indices) générées de façon régulière, parfois à haute fréquence, et sous différents formats (CSV, API, séries temporelles).

Pipeline de données : Un pipeline de données est une chaîne automatisée de traitements qui permet de faire circuler les données depuis leur source jusqu'à leur exploitation finale. Son rôle est d'assurer un traitement fiable, reproductible et structuré des données afin de les rendre exploitables pour l'analyse financière ou la modélisation.

ETL (Extract – Transform – Load) : L'ETL consiste à extraire les données depuis différentes sources, à les transformer (nettoyage, agrégation, mise en forme), puis à les charger dans un système de stockage. Contrairement à l'ELT, où les transformations se font après le chargement, l'ETL permet de contrôler la qualité des données en amont, ce qui est particulièrement important pour des données boursières sensibles.

Ingestion batch : L'ingestion batch repose sur le traitement de données par lots à des intervalles réguliers (par exemple, quotidiennement). Elle est bien adaptée aux données boursières historiques, plus simple à orchestrer et moins coûteuse que le streaming, mais moins réactive en temps réel.

Stockage et gouvernance des données : Les données peuvent être stockées dans un Data Lake (données brutes), un Data Warehouse (données structurées pour l'analyse) ou une approche hybride Lakehouse. La gouvernance repose sur la gestion des métadonnées, la traçabilité, la qualité et la sécurité des données.

Orchestration : L'orchestration correspond à la coordination des différentes tâches du pipeline via des workflows automatisés. Elle permet la planification, le suivi de l'exécution, la gestion des échecs et le redémarrage des traitements, garantissant la fiabilité globale du pipeline.

Notions financières essentielles : Les données boursières reposent principalement sur les prix, les volumes échangés et les séries temporelles. Leur analyse permet d'identifier des tendances, des comportements anormaux ou des anomalies de marché, essentielles pour la prise de décision financière.

1.2 Enjeux économiques et technologiques

Les données boursières sont devenues un élément stratégique central pour les institutions financières, car leur exploitation permet d'améliorer l'analyse des marchés, la gestion des risques et la prise de décision économique. Cette importance est confirmée par l'adoption massive des technologies analytiques dans le secteur financier : en 2025, plus de 95 % des banques mondiales ont intégré des solutions Big Data dans leurs opérations, et 85 % des banques américaines utilisent déjà ces technologies pour optimiser leurs processus et décisions.

La croissance exponentielle des volumes de données renforce encore ce besoin : les données numériques mondiales ont été multipliées par plus de 30 entre 2010 et 2020, atteignant 64 zettaoctets, et sont projetées à un peu plus de 180 zettaoctets en 2025, ce qui reflète l'ampleur des historiques financiers à gérer.

Pour gérer ces quantités massives, les approches traditionnelles basées sur des serveurs locaux et des traitements manuels deviennent rapidement inadéquates : elles manquent de scalabilité, sont coûteuses à maintenir, et peinent à automatiser le traitement ou à répondre aux besoins de performances en temps réel.

Le Big Data basé sur le cloud offre une alternative plus efficace : les plateformes de données cloud représentent déjà près de 60,5 % du marché Big Data en finance, car elles permettent de traiter de très grands volumes de données avec flexibilité, scalabilité et intégration native d'outils d'analyse avancés.

Par ailleurs, l'adoption du cloud dans les services financiers s'accélère : 60% des banques auront migré au moins 30% de leurs workloads critiques vers le cloud en 2025, améliorant l'efficacité opérationnelle, la réduction des coûts d'infrastructure et la résilience des systèmes.

Sur le plan économique, ces transformations se reflètent dans la croissance des investissements : le marché mondial du cloud computing était évalué à 676,29 milliards de dollars en 2024, avec une projection à 781,27 milliards de dollars en 2025, soulignant l'adoption croissante des technologies cloud pour supporter les infrastructures Big Data.

De même, le marché des solutions cloud financières est estimé à plus de 152 milliards USD en 2024, avec une croissance prévue soutenue, ce qui illustre l'importance de solutions évolutives, sécurisées et automatisées pour le traitement de données financières.

Ainsi, face à des volumes de données financiers toujours plus importants, l'intégration de solutions Big Data et cloud computing apparaît non seulement comme un levier technologique, mais aussi comme une nécessité économique pour maintenir la compétitivité, l'agilité et l'efficacité des institutions financières modernes.

2 | Description des données et méthodologie de conception

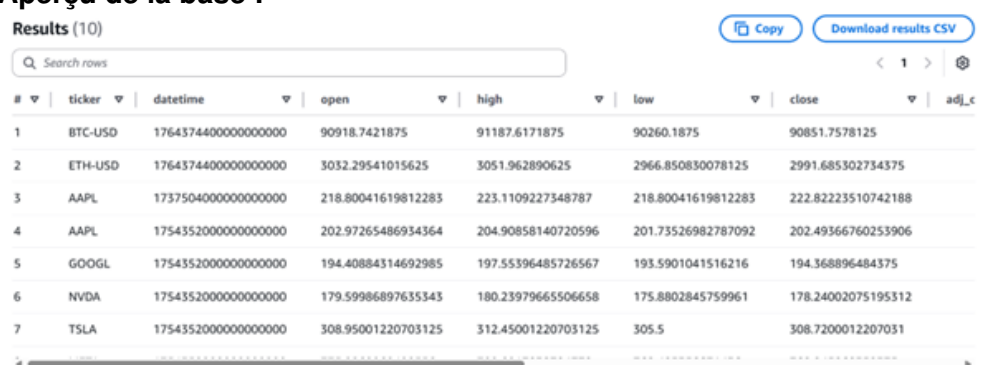
2.1 Description des données

Les données brutes sont structurées en 12 colonnes dans la base de stockage :

- ticker (string)
- datetime (bigint)
- open (double)
- high (double)
- low (double)
- close (double)
- adj_close (double)
- volume (double)
- source (string)
- ingested_at (string)
- run_id (string)
- data_date (string)

Nous expliciterons les plus importantes.

Aperçu de la base :



The screenshot shows a data table with 10 results. The table has columns: #, ticker, datetime, open, high, low, close, and adj_c. The data rows show financial information for BTC-USD, ETH-USD, AAPL, GOOGL, NVDA, and TSLA. The interface includes a search bar, a 'Copy' button, and a 'Download results CSV' button.

#	ticker	datetime	open	high	low	close	adj_c
1	BTC-USD	1764374400000000000	90918.7421875	91187.6171875	90260.1875	90851.7578125	
2	ETH-USD	1764374400000000000	3032.29541015625	3051.962890625	2966.850830078125	2991.685302734375	
3	AAPL	1737504000000000000	218.80041619812283	223.1109227348787	218.80041619812283	222.82223510742188	
4	AAPL	1754352000000000000	202.97265486934364	204.90858140720596	201.73526982787092	202.49366760253906	
5	GOOGL	1754352000000000000	194.40884314692985	197.55396485726567	193.5901041516216	194.368896484375	
6	NVDA	1754352000000000000	179.59986897635343	180.23979665506658	175.8802845759961	178.24002075195312	
7	TSLA	1754352000000000000	308.95001220703125	312.45001220703125	305.5	308.7200012207031	

2.1.1 Ticker

Un ticker est un symbole unique (combinaison de lettres et/ou de chiffres) utilisé pour identifier rapidement un titre financier (comme une action, un fonds ou une cryptomonnaie) sur une plateforme boursière.

2.1.2 Datetime

C'est l'index du DataFrame. Il représente la date et parfois l'heure exacte de chaque observation (jour, heure, minute selon l'intervalle demandé). Toute l'analyse temporelle repose dessus : rendements, moyennes mobiles, volatilité, etc. Sans cet index, la série perd son sens économique.

2.1.3 Open

Le prix **Open** est le premier prix auquel l'actif s'échange au début de la période considérée. Pour des données journalières, c'est le prix à l'ouverture du marché. Il capture souvent l'effet des informations arrivées hors séance (annonces macroéconomiques, résultats d'entreprises, chocs exogènes).

2.1.4 High

Le **High** correspond au prix le plus élevé atteint par l'actif pendant la période. Il donne une borne supérieure de la pression acheteuse. En analyse technique, il sert à identifier les niveaux de résistance, les amplitudes de volatilité et les comportements extrêmes intra-période.

2.1.5 Low

Le **Low** est le prix le plus bas observé sur la période. C'est la borne inférieure de la dynamique de prix, souvent interprétée comme un indicateur de stress ou de pression vendeuse. La combinaison *High-Low* mesure l'ampleur du mouvement (le "range").

2.1.6 Close

Le **Close** est le dernier prix de la période. C'est la valeur la plus utilisée en finance empirique : calcul de rendements, valorisations, backtests et modèles économétriques. Il est considéré comme le prix « révélateur » du consensus de marché à la fin de la période de cotation.

2.1.7 Volume

Le **Volume** mesure le nombre total de titres échangés pendant la période. Il ne donne pas le prix, mais l'intensité de l'activité. Un mouvement de prix accompagné d'un fort volume est généralement jugé plus crédible et significatif qu'un mouvement isolé à faible volume.

2.2 Méthodologie

Cette section détaille la méthodologie retenue pour concevoir et déployer un pipeline de données financières dans un environnement cloud. L'approche s'appuie sur une architecture de type **Data Lake**, associée à des services *serverless*, afin de garantir la scalabilité, la traçabilité et l'optimisation des coûts.

Elle couvre l'ensemble du cycle de vie des données : de leur ingestion automatisée depuis des sources externes jusqu'à leur exploitation analytique, en intégrant les étapes de transformation, de catalogage et de gouvernance des accès. Une attention particulière est accordée à la gestion des métadonnées et à l'application du principe du moindre privilège, assurant à la fois la fiabilité du pipeline et la sécurité des usages analytiques.

2.2.1 Conceptualisation et Choix d'Architecture

L'approche repose sur un modèle de Data Lake à plusieurs niveaux, implémenté sur **Amazon S3**, respectant la structuration classique en zones :

- **Zone Raw** : Stockage des données brutes, préservant l'intégrité de la source (format JSON/CSV natif).
- **Zone Curated** : Stockage des données nettoyées, transformées et optimisées pour l'analyse.

Le format de données **Parquet** a été choisi pour le stockage dans la zone Curated car il est la norme pour les charges de travail analytiques (OLAP) sur les data lakes du cloud. Il est conçu pour la rapidité des requêtes, l'efficacité du stockage (compression) et l'évolutivité.

Le choix d'un écosystème *Serverless* (**AWS Glue, Athena, Fargate**) minimise la gestion de l'infrastructure (NoOps), assurant une élasticité automatique et une facturation basée uniquement sur l'utilisation réelle.

2.2.2 Implémentation du Pipeline ETL

La méthodologie d'implémentation a suivi les étapes critiques suivantes :

Ingestion et Transformation (Extract & Transform)

L'ingestion des données est réalisée par l'exécution de requêtes en langage **Python**, s'appuyant sur la bibliothèque spécialisée `yfinance`. Ce module interagit via API avec les serveurs de Yahoo Finance afin de récupérer des données boursières structurées conformément aux paramètres définis dans le script d'extraction. Les informations ainsi obtenues sont conservées dans la zone Raw du Data Lake, garantissant la préservation intégrale des données sources pour assurer traçabilité et auditabilité.

Le processus de transformation a été implémenté via un service conteneurisé (**AWS Fargate/ECS**), conférant une flexibilité maximale dans le choix des langages et des bibliothèques pour les opérations de nettoyage et d'enrichissement. La tâche principale de

cette phase est de convertir les données brutes ingérées dans la zone Raw vers le format optimisé (Parquet) et de les déposer dans la zone Curated.

Catalogage et Indexation (AWS Glue)

Une fois les données dans la zone Curated, l'étape cruciale de la découverte et de la schématisation est réalisée par un **Crawler AWS Glue**.

- **Rôle du Crawler** : Analyser les fichiers Parquet dans le chemin S3 renseigné, déduire le schéma, et enregistrer les métadonnées (nom des colonnes, types de données) dans le **AWS Glue Data Catalog**.
- **Partitionnement** : La bonne pratique du partitionnement (via une colonne telle que `data_date`) est implicitement adoptée pour optimiser les coûts et les performances des requêtes en limitant le volume de données scannées.

Accès Analytique (Amazon Athena)

Le moteur d'interrogation analytique sans serveur, **Amazon Athena**, est utilisé pour l'accès aux données.

- **Rôle d'Athena** : Exécuter des requêtes SQL standard directement sur les fichiers S3 en utilisant le schéma fourni par Glue.
- **Configuration Essentielle** : La méthodologie exige la définition d'un emplacement S3 spécifique pour le stockage des résultats intermédiaires, garantissant la traçabilité et le bon fonctionnement du service.

2.2.3 Gouvernance des Données et Sécurité (IAM)

L'aspect le plus rigoureux de la méthodologie concerne la gestion des accès et le principe du moindre privilège (*Principle of Least Privilege* - PoLP), même lorsqu'un accès large est requis. La stratégie repose sur les rôles et les politiques IAM :

- **Création du Groupe** `yf-etl-user` : Ce groupe centralise les permissions pour les utilisateurs consommateurs (Analystes, Data Scientists).
- **Politique IAM Personnalisée** : Une politique détaillée (`FinanceETLTeamPolicy`) est attachée à ce groupe, autorisant des actions spécifiques sur des ressources limitées (*ARN-level access control*).
- **Accès Lecture/Écriture Contrôlé (S3)** : Autorisation d'accès aux fichiers uniquement dans les zones `curated/` et `athena-results/`.
- **Contrôle Strict du Moteur (ECS/Fargate)** : L'ajout optionnel des permissions `ecs:RunTask` est strictement limité à l'ARN de la Définition de Tâche spécifique, empêchant l'exécution ou l'arrêt de services critiques non liés.
- **Principe de Deny Explicite** : L'utilisation d'une clause `Deny` explicite pour interdire la suppression (`s3:DeleteObject`) des données dans les zones Raw ou Processed protège le Data Lake contre les erreurs humaines accidentelles.

2.2.4 Phase de Validation et de Visualisation

La méthodologie se termine par la validation de l'intégralité de la chaîne et l'ouverture à la consommation :

- **Validation Fonctionnelle** : L'exécution de requêtes SQL simples dans Athena sert de test de bout en bout pour confirmer que le schéma Glue est valide et que les données S3 sont lisibles.
- **Correction de Méta-erreurs** : La détection d'erreurs de métadonnées (ex : `HIVE_INVALID_METADATA: duplicate columns`) a nécessité une correction directe du schéma dans la console AWS Glue, soulignant l'importance de la gestion active du catalogue.
- **Consommation (Dash)** : Le pipeline est prêt pour la phase finale de Business Intelligence ou d'analyse avancée via un outil comme **Dash** (Python), qui se connectera directement aux tables définies dans Athena/Glue.

En résumé, la méthodologie mise en œuvre est celle d'un Data Lake évolutif et administré par des métadonnées, avec un accent fort sur la granularité des permissions IAM pour ségréguer les responsabilités entre les ingénieurs (gestion du pipeline) et les analystes (consommation des données).

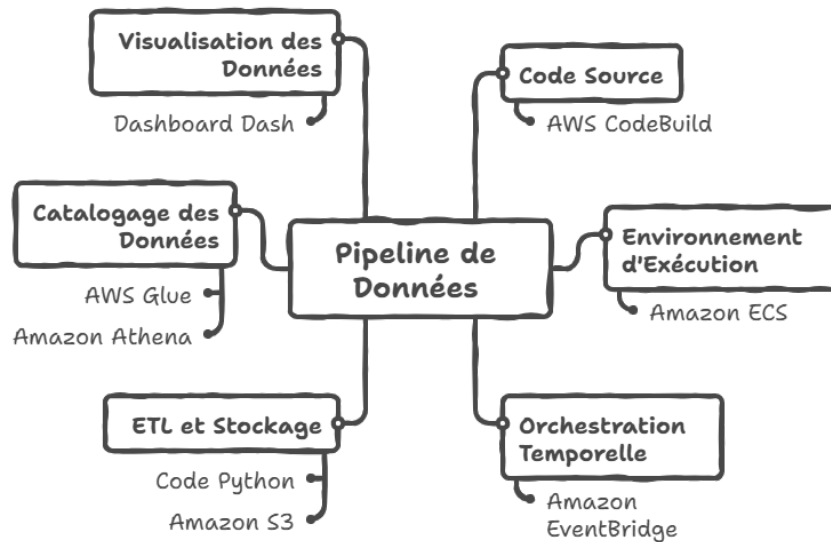
3 | Architecture globale du pipeline

3.1 Présentation du schéma général

L'architecture de notre pipeline de données repose sur une chaîne d'intégration continue (CI/CD) couplée à une exécution conteneurisée, aboutissant à une couche d'analyse et de visualisation interactive.

Le schéma général illustre le flux complet :

1. Le code source sur **GitHub** est récupéré et construit par **AWS CodeBuild**.
2. L'environnement d'exécution (Image Docker) est déployé sur **Amazon ECS** (Elastic Container Service).
3. L'orchestration temporelle est gérée par **Amazon EventBridge** qui déclenche les tâches.
4. Le code Python exécuté gère l'ETL et stocke les résultats sur **Amazon S3**.
5. Les données sont cataloguées par **AWS Glue** et rendues accessibles via **Amazon Athena**.
6. Enfin, un Dashboard **Dash** consomme ces données pour la visualisation finale.



Made with Napkin

3.2 Description des flux

Le cycle de vie de la donnée est piloté par le code pour l'ingestion, puis par des services managés pour l'exposition et l'analyse.

3.2.1 Composants principaux

1. Couche de construction et d'ingestion

Cette couche assure la préparation de l'environnement et l'acquisition des données.

- **Construction (AWS CodeBuild)** : Ce service récupère le code depuis GitHub, construit l'image Docker avec toutes les dépendances nécessaires, et prépare l'environnement pour ECS.
- **Exécution (Amazon ECS)** : Une tâche ECS lance le conteneur qui exécute le script Python.
- **Logique ETL** : Toute la logique d'Extraction, de Transformation et de Chargement est centralisée dans le code Python, offrant une maîtrise totale sur les traitements.

2. Couche de stockage structuré (S3)

L'organisation dans le Data Lake (**Amazon S3**) est gérée par le code Python qui segmente les données en deux zones :

- **Raw (brute)** : Le dossier `/raw` reçoit les données telles qu'elles sont extraites directement mises sous format parquet.
- **Curated (Raffinée)** : Le dossier `/curated` contient les données nettoyées, divisées dynamiquement en sous-dossiers `/intervalle-1-d` et `/intervalle-1-h` selon la granularité temporelle.

3. Couche de catalogage et de requêtage

Pour rendre les fichiers du dossier `curated` exploitables comme une base de données :

- **Catalogage (AWS Glue Crawler)** : Nous avons configuré un Crawler Glue qui inspecte le dossier `/curated`.
 - *Fonctionnement* : Lors de la première exécution, il scanne tout l'historique pour créer le schéma initial. Lors des exécutions suivantes, il détecte uniquement les nouvelles partitions (nouvelles données) pour mettre à jour le catalogue.
- **Requêtage (Amazon Athena)** : Grâce au catalogue Glue, Athena nous permet d'interroger directement les fichiers S3 en utilisant du SQL standard, sans gérer de serveur de base de données.

4. Couche d'orchestration

- **Planification (Amazon EventBridge)** : L'automatisation du pipeline est assurée par Amazon EventBridge. Des règles planifiées (Cron) déclenchent l'exécution des tâches ECS à des intervalles réguliers (quotidiennement à la clôture des marchés : 16h , heure de New York) pour assurer la mise à jour des données.

5. Couche de visualisation

- **Dashboard (Dash)** : L'interface utilisateur finale est une application web développée avec le framework Python **Dash**. Elle se connecte à Athena pour récupérer les données agrégées et afficher les indicateurs financiers.

3.3 Choix du fournisseur cloud

Bien qu'Azure et Google Cloud Platform (GCP) offrent des services équivalents, AWS a été sélectionné pour sa maturité sur le Data Lake et son intégration fluide entre les conteneurs (ECS) et l'analytique serverless (Athena/Glue).

L'objectif était d'assurer l'évolutivité et l'optimisation des coûts (modèle Pay-as-you-go) tout en gardant la main sur le code ETL.

L'architecture repose sur les services AWS suivants :

- **AWS CodeBuild** : Pour la CI/CD et la construction de l'image Docker.
- **Amazon ECS** : Pour l'exécution des scripts ETL.
- **Amazon S3** : Pour le stockage hiérarchisé (raw, curated).
- **Amazon EventBridge** : Pour la planification temporelle et l'orchestration des tâches.
- **AWS Glue (Crawler)** : Pour la détection de schéma et le catalogage.
- **Amazon Athena** : Pour l'interface SQL serverless.

4 | Limites, risques et évolutions potentielles

4.1 Limites actuelles de la solution

4.1.1 Limites liées à la granularité du pipeline de données

L'architecture actuelle du pipeline ETL repose sur un mode de traitement par lots, ce qui limite fortement la capacité du système à ingérer et traiter les données au fil de l'eau. Dans cette configuration, les variations de prix ne peuvent être prises en compte qu'à l'issue de l'exécution du job ETL, avec une latence dépendante de sa fréquence : à savoir 24 heures. Cette contrainte impacte directement la fiabilité temporelle des analyses produites, le pipeline n'étant pas en mesure de refléter des évolutions de marché survenues entre deux exécutions successives.

4.1.2 Risques liés à la dérive du schéma des données

Un autre point critique concerne la gestion de la qualité des données et du schéma. Toute modification non contrôlée de la structure des données à la source, telle que l'ajout ou la suppression de colonnes ou un changement de type, peut entraîner une dérive du schéma dans le catalogue AWS Glue. Ce phénomène peut provoquer l'échec des jobs ETL ainsi que des requêtes Athena, compromettant ainsi la disponibilité et la fiabilité du pipeline de données dans son ensemble.

4.1.3 Limites de performance et d'optimisation de l'ETL

Les performances et les coûts du pipeline ETL constituent également une limite notable. Le job de transformation, qu'il soit exécuté via AWS Glue ou un cluster EMR, n'est pas nécessairement optimisé pour tirer pleinement parti du moteur de calcul distribué. Actuellement, certains calculs d'indicateurs techniques, tels que les moyennes mobiles, sont réalisés côté client dans l'application Dash à l'aide de Pandas. Cette approche alourdit la charge applicative et conduit à interroger des volumes de données plus importants que nécessaire via Athena, impactant à la fois les performances et les coûts.

4.1.4 Manque d'observabilité et de supervision opérationnelle

Enfin, l'absence de mécanismes de surveillance centralisée constitue une limite opérationnelle importante. Sans observabilité renforcée, il devient difficile d'identifier de manière proactive les échecs des jobs ETL, les problèmes de saturation des ressources ou les goulots d'étranglement en matière de performance. Cette situation complique la maintenance du pipeline et augmente le risque d'incidents non détectés impactant la disponibilité des données.

4.2 Perspectives d'évolution de la solution

4.2.1 Évolution vers une architecture quasi temps réel

Afin de répondre aux limites liées à la granularité du pipeline, une évolution vers une architecture quasi temps réel constitue une perspective majeure. La migration vers des services d'ingestion événementielle tels qu'AWS Kinesis ou Amazon Managed Streaming for Kafka permettrait de traiter les mises à jour de prix en continu. Cette approche améliorerait significativement la fiabilité temporelle des données et renforcerait la capacité du système à refléter rapidement les évolutions du marché.

4.2.2 Renforcement de la robustesse du schéma et de la qualité des données

Pour limiter les risques de dérive du schéma, la mise en place de tests de contrat de données apparaît comme une évolution essentielle. L'intégration de validations automatiques du schéma des fichiers Parquet dans une chaîne CI/CD, par exemple via AWS CodeBuild, permettrait de garantir la conformité des données avant toute mise à jour du catalogue Glue. Cette démarche contribuerait à sécuriser les jobs ETL et les requêtes Athena face aux évolutions des sources amont.

4.2.3 Optimisation des performances et des coûts du pipeline ETL

Une amélioration significative des performances pourrait être obtenue en déplaçant la logique de calcul des indicateurs techniques vers le job ETL lui-même. L'implémentation des calculs de moyennes mobiles directement en PySpark au sein de la couche de transformation permettrait de tirer pleinement parti du moteur de calcul distribué. Cette optimisation réduirait la charge côté application Dash et limiterait les volumes de données interrogés par Athena, améliorant ainsi les performances globales et la maîtrise des coûts.

4.2.4 Mise en place d'une observabilité complète du pipeline

Enfin, l'amélioration de l'observabilité du pipeline représenterait un levier important pour la fiabilité opérationnelle. La centralisation des logs d'exécution des jobs ETL dans Amazon CloudWatch, associée à la création d'alarmes sur les métriques critiques de

performance et de consommation de ressources, permettrait une détection proactive des anomalies. Cette évolution faciliterait la maintenance, réduirait les temps de résolution des incidents et renforcerait la stabilité globale de la solution.

Conclusion

Ce projet a permis de concevoir et de mettre en œuvre une pipeline de données cloud complète, automatisée et scalable, dédiée au traitement de données financières. En s'appuyant sur une architecture **Data Lake** et sur des services managés d'**Amazon Web Services** tels qu'**Amazon S3**, **Amazon ECS** avec **AWS Fargate**, **AWS Glue**, **Amazon Athena**, **Amazon EventBridge**, **AWS IAM** et **Amazon CloudWatch**, nous avons construit une solution couvrant l'ensemble du cycle de vie de la donnée, depuis son ingestion jusqu'à sa restitution analytique.

Le choix des données boursières comme cas d'usage s'est révélé particulièrement pertinent. Par leur nature temporelle, leur volatilité et leur sensibilité aux délais de traitement, ces données constituent un terrain d'application exigeant pour les architectures Big Data. Le pipeline mis en place répond à ces contraintes en garantissant une ingestion automatisée en mode batch, une transformation contrôlée, un stockage structuré et optimisé, ainsi qu'un accès analytique flexible via des requêtes SQL serverless. L'organisation des données en zones Raw et Curated, combinée au catalogage automatique des métadonnées, assure traçabilité, qualité et gouvernance des données.

L'intégration d'une chaîne **CI/CD** et d'une orchestration temporelle renforce la robustesse opérationnelle de la solution. La conteneurisation des traitements ETL, déclenchés automatiquement selon une planification définie, permet une exécution reproductible et indépendante de l'infrastructure sous-jacente. Cette approche illustre pleinement les bénéfices du cloud computing en termes d'élasticité, de réduction des coûts et de simplification de l'exploitation.

La valorisation finale des données à travers un tableau de bord interactif développé avec **Dash** constitue l'aboutissement logique du pipeline. Elle démontre concrètement comment des données financières brutes peuvent être transformées en indicateurs visuels et exploitables, facilitant l'analyse des tendances de marché, des volumes échangés et de l'évolution des prix. Cette couche de visualisation met en évidence la capacité du pipeline à soutenir des usages analytiques et décisionnels réels.

Au-delà de l'aspect technique, ce projet met en lumière le rôle stratégique des pipelines de données dans les environnements financiers modernes. Une architecture

cloud bien conçue constitue un socle essentiel pour l'analyse, le reporting et la prise de décision basée sur les données, tout en offrant des garanties en matière de sécurité, de supervision et d'évolutivité. Enfin, les perspectives d'évolution identifiées — intégration de flux quasi temps réel, optimisation des traitements analytiques ou renforcement de l'observabilité — confirment la pertinence et la durabilité de l'approche adoptée.

Bibliographie

- astera Astera. *Build a Data Pipeline*.
Disponible sur : <https://www.astera.com/type/blog/build-a-data-pipeline/>
- snaplogic SnapLogic. *Data Pipelines : What They Are and How to Build One From Scratch*.
Disponible sur : <https://www.snaplogic.com/fr/blog/data-pipelines-what-they-are-and-how>
- yfinance Aroussi, Ran. *yfinance : Yahoo! Finance market data downloader*.
Disponible sur : <https://ranaroussi.github.io/yfinance/>
- idc International Data Corporation (IDC). (2021). *Global DataSphere Forecast*.
Disponible sur : <https://www.idc.com/getdoc.jsp?containerId=prUS47560321>
- coinlaw1 CoinLaw. (2024). *Big Data in Finance Statistics*.
Disponible sur : <https://coinlaw.io/big-data-in-finance-statistics/>
- coinlaw2 CoinLaw. (2024). *Cloud Computing in Financial Services Statistics*.
Disponible sur : <https://coinlaw.io/cloud-computing-in-financial-services-statistics/>
- fortune Fortune Business Insights. (2024). *Cloud Computing Market Size*.
Disponible sur : <https://www.fortunebusinessinsights.com/cloud-computing-market-102697>